



Atividade A4: Desenvolvimento de Repositórios Guiado por Testes (TDD)



Objetivo Nesta atividade, você deverá implementar novas funcionalidades na interface `VeterinarioRepository`, recomendado utilizar o ciclo TDD (Test-Driven Development).

Regra de Ouro:

1. Não é permitido escrever nenhum método no repositório sem que haja um teste falhando para ele primeiro.
2. É obrigatório utilizar o padrão AAA (Arrange, Act, Assert).
3. Você pode (e deve) modificar o arquivo `import t.sql` para criar o seu próprio cenário de testes.



Valor: 15 pontos



Entrega: Link do GitHub ou projeto compactado.



O Desafio

Você deve evoluir o sistema da clínica veterinária seguindo os ciclos abaixo. Para cada funcionalidade, siga o rito:

1. **RED:** Escreva o teste na classe `VeterinarioRepositoryTests`, tente executar e veja-o falhar.
2. **GREEN:** Implemente o método/query necessária no `VeterinarioRepository` para o teste passar.
3. **REFACTOR:** Melhore o código ou a query se necessário.



Ciclo 1: Busca por Nome (Exato e Case Insensitive)

Requisito: O sistema deve buscar um veterinário pelo nome:

- **Teste 1 (RED):** (Exato) Tente buscar um nome existente usando letras minúsculas (ex: "pedro") quando no banco está "PEDRO". O teste deve falhar. Ou seja, busca por valor exato. Testar um nome que exista também.
- **Teste 2 (RED):** (Exato) Tente buscar um nome existente usando letras minúsculas (ex: "PEDRO") quando no banco está "PEDRO". O teste deve retornar o veterinário correspondente.
- **Teste 3 (RED):** (case insensitive) Tente buscar um nome existente usando letras minúsculas (ex: "jose") quando no banco não existe veterinário com esse nome. O teste deve falhar.

- **Teste 4 (RED):** (case insensitive) Tente buscar um nome existente usando letras minúsculas(ex: “joao”) quando no banco existe veterinário com esse nome em caixa alta. O teste retorna o veterinário
- **Ação (GREEN):** Crie os métodos `findByName` com e sem o query methods : `IgnoreCase`

Ciclo 2: Busca por "Pedaço" do Nome (LIKE)

Requisito: A clínica precisa retornar uma lista de veterinários que contenham um termo específico no nome.

- **Teste 1 (RED):** Crie um teste que busca pela sílaba "ro" e espera receber "Pedro" e "Roberto" (certifique-se de que esses dados existam no seu `import.sql`). O teste deve falhar.
- **Teste 2 (RED):** Valide o comportamento caso o parâmetro de busca seja uma String vazia (deve retornar todos os veterinários).
- **Ação (GREEN):** Implemente um método que atende a solicitação.

Ciclo 3: Filtros de Salário

Requisito: O setor financeiro precisa de três novas formas de filtrar os veterinários pelo salário.

- **Cenário A:** Veterinários com salário **superior** a um valor informado.
- **Cenário B:** Veterinários com salário **inferior** a um valor informado.
- **Cenário C:** Veterinários com salário dentro de uma **faixa de valores** (mínimo e máximo).
- **Instrução TDD:** Escreva um teste para cada cenário antes de criar os métodos. Explore a convenção de nomes do Spring Data (ex: `findBySalarioGreaterThan`, `findBySalarioLessThan`, `findBySalarioBetween`) ou crie as `@Query` manualmente.

Ciclo 4: Filtros de Data de Nascimento (ou Contratação)

Requisito: O RH precisa buscar veterinários em determinados períodos de data (ex: data de nascimento).

- **Teste (RED):** Crie um teste que define uma `dataInicio` e uma `dataTermino` (você pode utilizar `Instant.parse("2000-01-01T00:00:00Z")` e `Instant.now()`) e verifique se os veterinários retornados estão dentro desse intervalo.
- **Ação (GREEN):** Crie o método correspondente no repositório. Dica: você não precisa escrever a `@Query` se usar a nomenclatura correta do Spring, como `findByDataNascimentoBetween`.

Ciclo 5: O Teste de "Fogo" (Update)

Requisito: Validar se a atualização de dados está persistindo corretamente no banco.

- **Teste:**

1. Busque um veterinário existente.
2. Altere seu nome, salário e data no objeto.
3. Salve (`repository.save(veterinario)`).
4. **Desafio TDD:** Utilize os métodos de busca criados nos ciclos anteriores para verificar se o veterinário agora só é encontrado pelos **novos dados** informados, e não mais pelos antigos.

Ciclo 6: Estatísticas Rápidas (Contagem)

Requisito: O painel administrativo da clínica precisa exibir rapidamente quantos veterinários ganham acima do teto salarial (ex: R\$ 5.000,00).

- **Teste (RED):** Verifique no seu `import.sql` quantos veterinários ganham mais de 5.000. Escreva um teste esperando exatamente esse número (ex: `assertEquals(3, resultado)`). O teste vai falhar.
- **Ação (GREEN):** Crie o método utilizando o prefixo `countBy...` (Ex: `countBySalarioGreaterThan(Double valor)`).

Ciclo 7: Teste de Restrição (Unique Constraint)

Requisito: O banco de dados não deve aceitar dois veterinários com o mesmo e-mail (caso o mapeamento da entidade tenha `@Column(unique = true)`).

- **Teste:**
 1. Busque um veterinário existente e pegue seu e-mail.
 2. Instancie um *novo* objeto `Veterinario` setando esse mesmo e-mail.
 3. Tente dar um `repository.save()` e depois um `repository.flush()`.
 4. **Desafio TDD:** Envolve a chamada em um `assertThrows` esperando uma `DataIntegrityViolationException`. Isso prova que a camada de persistência está blindada contra dados duplicados.

Observações Importantes:

- Utilize o banco em memória **H2** e o arquivo `import.sql` para garantir que o cenário (Arrange) seja sempre o mesmo a cada execução.
- Fiquem à vontade para criar outros testes além dos sugeridos. O mais importante é demonstrar que você pensou no teste antes da implementação.

- Vou deixar o link do projeto base que estamos usando em aula, porém nada impede que vocês usem outro projeto. Neste caso, peço que disponibilizem uma descrição básica do sistema.